

Laporan Tugas Kecil I

Mata Kuliah IF2211 Strategi Algoritma



Disusun oleh :

13524112 - Richard Samuel Simanullang
K2

1. Penjelasan Algoritma Brute Force yang Dipakai

1.1 Algoritma iteratif (solver.java)

Algoritma berikut menempatkan queen berdasarkan kemungkinan kombinasi penempatan ratu. Algoritma ini menempatkan ratu berdasar pada aturan baris, kolom, dan bertentangan dimulai dari mengisi ratu pada tiap kolom. Berikut tahap pengerjaan algoritma ini :

1. Hitung total kemungkinan kombinasi n^n .
2. Lakukan loop dari 0 sampai total kemungkinan - 1.
3. Dalam setiap iterasi, diubah iteratornya jadi koordinat baris untuk setiap queen di masing-masing kolom.
4. Lakukan pengecekan validitas koordinat queen (cekVal) setelah posisi seluruh ratu terbentuk. Fungsi ini memeriksa apakah ada pelanggaran aturan baris (satu baris dua queen), aturan warna (warna yang sama), dan ada disekitarnya.
5. Jika konfigurasi valid ditemukan utk pertama kali, simpan konfigurasi tersebut sebagai solusi.
6. Lanjutkan iterasi hingga hitungan ke- n^n selesai.

Penjelasan lanjut : Algoritma ini brute force murni karena mencari semua solusi tanpa ada pruning, Iterasi yang mengikuti aturan permainan bukan panduan baru, serta menyimpan solusi setelah iterasi semua kemungkinan.

1.2 Algoritma dengan pendekatan warna (opt_solver.java)

Algoritma berikut menempatkan queen berdasarkan warna terlebih dahulu. Algoritma ini mengumpulkan koordinat pada board berdasarkan warnanya dalam list_warna. Kemudian mencari semua kemungkinan pada warna tersebut, lalu berjalan rekursif untuk warna selanjutnya. Langkah selengkapnya seperti berikut :

1. Kelompokkan semua koordinat board berdasarkan warnanya ke dalam list (list_Warna).
2. Mulai fungsi rekursif dari indeks warna pertama.
3. tiap langkah rekursi:
 - Ambil daftar posisi petak yang ada untuk warna saat ini.
 - Iterasi melalui setiap kemungkinan posisi petak untuk warna tersebut.
 - Pilih satu posisi, lalu panggil fungsi rekursif untuk warna berikutnya (indexWarna + 1).
4. Base Case : Kalau semua warna sudah memiliki posisi Queens (indexWarna == jumlah warna), baru dicek validitas (cekVal).

5. Fungsi cekVal memeriksa sesuai aturan game.
6. Jika valid, tandai found_sol = true dan simpan solusi.

Penjelasan lanjut : Pendekatan warna ini menelusuri semua kemungkinan kotak yang ada pada tiap warna karena itu saya menganggapnya brute force murni, namun jika rekursi dan backtracking dianggap tidak murni dalam konteks solver ini, maka algoritma ini dapat dijadikan sebagai optimisasi.

2. Source Code

Berikut Source Code terkait implementasi Solver* :

```
main.java
import java.util.*;
import java.io.*;

class Pos {
    int r, c;
    Pos(int r, int c) {
        this.r = r;
        this.c = c;
    }
}

public class main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan nama file input: ");
        String filePath = "test/" + input.nextLine().trim();
        input.close();

        List<String> baris = new ArrayList<>();
        try {
            Scanner sc = new Scanner(new File(filePath));
            while (sc.hasNextLine()) {
                String line = sc.nextLine().trim();
                if (!line.isEmpty()) {
                    baris.add(line);
                }
            }
            sc.close();
        } catch (FileNotFoundException e) {
```

```

        System.out.println("File tidak ditemukan: " + filePath);
        return;
    }

    int n = baris.size();
    char[][] papan = new char[n][n];
    List<Pos>[] list_Warna = new ArrayList[26];
    for (int i = 0; i < 26; i++) {
        list_Warna[i] = new ArrayList<>();
    }

    for (int i = 0; i < n; i++) {
        String row = baris.get(i);
        if (row.length() != n) {
            System.out.println("Panjang baris ke-" + i + " tidak sama dengan " + n);
            return;
        }
        for (int j = 0; j < n; j++) {
            char ch = row.charAt(j);
            if (ch < 'A' || ch > 'Z') {
                System.out.println("Ada input warna tidak valid");
                return;
            }
            papan[i][j] = ch;
            int idx = ch - 'A';
            list_Warna[idx].add(new Pos(i, j));
        }
    }

    // opt_solve opt_solver = new opt_solve(papan, n, list_Warna);
    solve solver = new solve(papan, n);
    boolean ditemukan = solver.cariSolusi();
    // boolean ditemukan = opt_solver.cariSolusi();
    if (ditemukan) {
        solver.tampilkanSolusi();
        // opt_solver.viewSol();
    } else {
        System.out.println("\nTidak ada solusi yang ditemukan");
    }
}
}

```

solve.java

```
import java.util.*;

public class solve {
    private char[][] papan;
    private int n;
    private long iter = 0;
    private int[] solusi;
    private boolean dapat_Solusi = false;

    // buat GUI live update
    public interface SolveListener {
        void onTry(int[] letak, int n, long iterasi);
        void onSolved(int[] solusi, int n, long iterasi);
    }

    private SolveListener listener;
    private long lastUpdateTime = 0;
    private static final long intervalms = 30; //bisa diedit agar lama live updatenya

    public void setListener(SolveListener listener) {
        this.listener = listener;
    }

    public solve(char[][] papan, int n) {
        this.papan = papan;
        this.n = n;
        this.solusi = new int[n];
    }

    public boolean cariSolusi() {
        long startTime = System.currentTimeMillis();
        int[] letak = new int[n];
        bruteForce(letak, 0);
        long endTime = System.currentTimeMillis();
        System.out.println("\nWaktu pencarian: " + (endTime - startTime) + " ms");
        System.out.println("Banyak kasus yang ditinjau: " + iter + " kasus");
        return dapat_Solusi;
    }

    //getttter
    public int[] getSolusi() {
```

```

        return solusi;
    }

    public long getIter() {
        return iter;
    }

    private void bruteForce(int[] koord_Queen, int col) {
        long jlh_kom = 1;
        for (int i = 0; i < n; i++) {
            jlh_kom *= n;
        }
        for (long i = 0; i < jlh_kom; i++) {
            long temp = i;
            for (int i_col = 0; i_col < n; i_col++) {
                koord_Queen[i_col] = (int) (temp % n);
                temp /= n;
            }
            iter++;
            // tiap coba kemungkinan, langsung ke main buat tampilan di GUI
            if (listener != null) {
                long now = System.currentTimeMillis();
                if (now - lastUpdateTime >= intervalms) {
                    lastUpdateTime = now;
                    listener.onTry(koord_Queen.clone(), n, iter);
                    try { Thread.sleep(1); }
                    catch (InterruptedException e) { Thread.currentThread().interrupt(); }
                }
            }
            if (cekVal(koord_Queen)) {
                if (!dapat_Solusi) {
                    dapat_Solusi = true;
                    solusi = koord_Queen.clone();
                    if (listener != null) {
                        listener.onSolved(solusi.clone(), n, iter);
                    }
                }
            }
        }
    }

    private boolean cekVal(int[] koord_Queen) {

```

```

for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        int row1 = koord_Queen[i], col1 = i;
        int row2 = koord_Queen[j], col2 = j;
        // baris
        if (row1 == row2) return false;
        // warna
        if (papan[row1][col1] == papan[row2][col2]) return false;
        // sekitar
        if (Math.abs(row1 - row2) <= 1 && Math.abs(col1 - col2) <= 1) {
            return false;
        }
    }
}
return true;
}

public void tampilkanSolusi() {
    if (!dapat_Solusi) {
        System.out.println("\nTidak ada solusi!");
        return;
    }

    System.out.println("\nSolusi ditemukan.");
    char[][] hasil = new char[n][n];

    for (int i = 0; i < n; i++) hasil[i] = papan[i].clone();
    for (int col = 0; col < n; col++) hasil[solusi[col]][col] = '#';

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) System.out.print(hasil[i][j]);
        System.out.println();
    }
}
}

```

opt_solve.java

```
import java.util.*;

public class opt_solve {
    private char[][] papan;
    private int n;
    private List<Pos>[] list_Warna;
    public List<Character> akt;
    private long iter = 0;
    private int[] pos;
    private boolean found_sol = false;
    public int[] best;

    // live update GUI
    public interface SolveListener {
        void onTry(int[] currentPos, List<Character> akt, List<Pos>[] list_Warna, long iterasi);
        void onSolved(int[] bestPos, List<Character> akt, List<Pos>[] list_Warna, long iterasi);
    }

    private SolveListener listener;
    private long lastUpdateTime = 0;
    private static final long UPDATE_INTERVAL_MS = 30; //bisa diedit

    public void setListener(SolveListener listener) {
        this.listener = listener;
    }

    public opt_solve(char[][] papan, int n, List<Pos>[] list_Warna) {
        this.papan = papan;
        this.n = n;
        this.list_Warna = list_Warna;
        this.akt = new ArrayList<>();
        for (int i = 0; i < 26; i++) {
            if (!list_Warna[i].isEmpty()) {
                akt.add((char)('A' + i));
            }
        }
        this.pos = new int[akt.size()];
        this.best = new int[akt.size()];
    }

    public boolean cariSolusi() {
```



```

    long startTime = System.currentTimeMillis();
    bruteForce(0);
    long endTime = System.currentTimeMillis();
    System.out.println("\nWaktu pencarian: " + (endTime - startTime) + " ms");
    System.out.println("Banyak kasus yang ditinjau: " + iter + " kasus");
    return found_sol;
}

```

```

public long getI() {
    return iter;
}

```

```

private void bruteForce(int indexWarna) {
    if (indexWarna == akt.size()) {
        iter++;
        if (listener != null) {
            long now = System.currentTimeMillis();
            if (now - lastUpdateTime >= UPDATE_INTERVAL_MS) {
                lastUpdateTime = now;
                listener.onTry(pos.clone(), akt, list_Warna, iter);
                try { Thread.sleep(1); }
                catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            }
        }
        if (cekVal()) {
            found_sol = true;
            for (int i = 0; i < akt.size(); i++) {
                best[i] = pos[i];
            }
            if (listener != null) {
                listener.onSolved(best.clone(), akt, list_Warna, iter);
            }
        }
        return;
    }
}

```

```

char warna = akt.get(indexWarna);
int idx = warna - 'A';
List<Pos> posisi_Warna = list_Warna[idx];
for (int i = 0; i < posisi_Warna.size(); i++) {
    pos[indexWarna] = i;
    bruteForce(indexWarna + 1);
}

```

```

        if (found_sol) {
            return;
        }
    }
}

private boolean cekVal() {
    List<Pos> posisi_Queens = new ArrayList<>();
    for (int i = 0; i < akt.size(); i++) {
        char warna = akt.get(i);
        int idx = warna - 'A';
        int posIdx = pos[i];
        Pos pos = list_Warna[idx].get(posIdx);
        posisi_Queens.add(pos);
    }
    for (int i = 0; i < posisi_Queens.size(); i++) {
        for (int j = i + 1; j < posisi_Queens.size(); j++) {
            Pos q1 = posisi_Queens.get(i);
            Pos q2 = posisi_Queens.get(j);
            if (q1.r == q2.r) {
                return false;
            }
            if (q1.c == q2.c) {
                return false;
            }
            int deltaRow = Math.abs(q1.r - q2.r);
            int deltaCol = Math.abs(q1.c - q2.c);
            if ((deltaRow == 1 && deltaCol == 0) ||
                (deltaRow == 0 && deltaCol == 1) ||
                (deltaRow == 1 && deltaCol == 1)) {
                return false;
            }
        }
    }

    return true;
}

public void tampilkanSolusi() {
    if (!found_sol) {
        System.out.println("\nTidak ada solusi!");
        return;
    }
}

```

```

    }
    System.out.println("\nSolusi ditemukan.");
    char[][] hasil = new char[n][n];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            hasil[i][j] = papan[i][j];
        }
    }
    for (int i = 0; i < akt.size(); i++) {
        char warna = akt.get(i);
        int idx = warna - 'A';
        int posIdx = best[i];
        Pos pos = list_Warna[idx].get(posIdx);
        hasil[pos.r][pos.c] = '#';
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            System.out.print(hasil[i][j]);
        }
        System.out.println();
    }
}
}

```

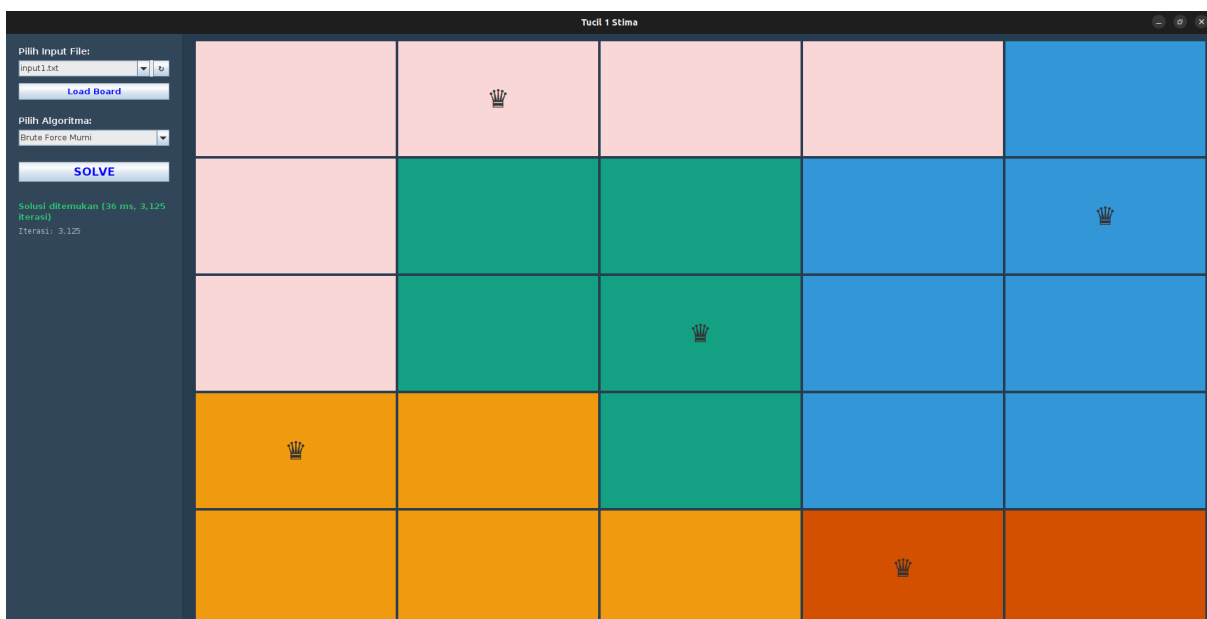
*Untuk implementasi GUI serta Background thread/UI thread sepenuhnya terlampir pada github.

3. Hasil input dan output program

3.1 TC 1

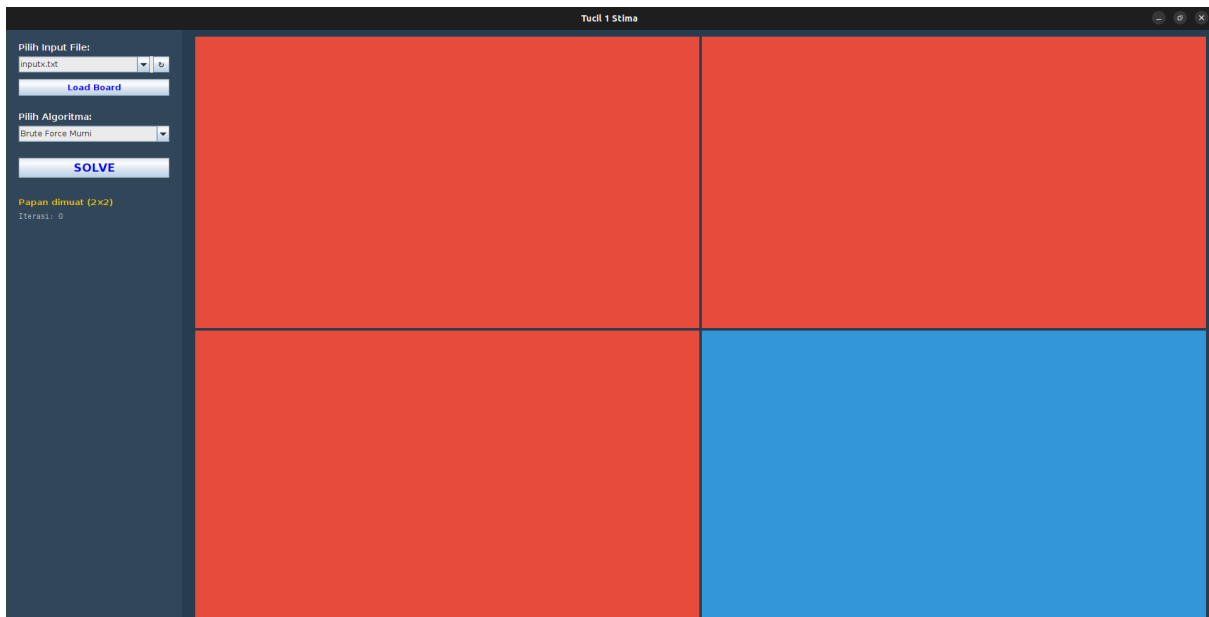


Gambar 3.1.1 Input Testcase normal

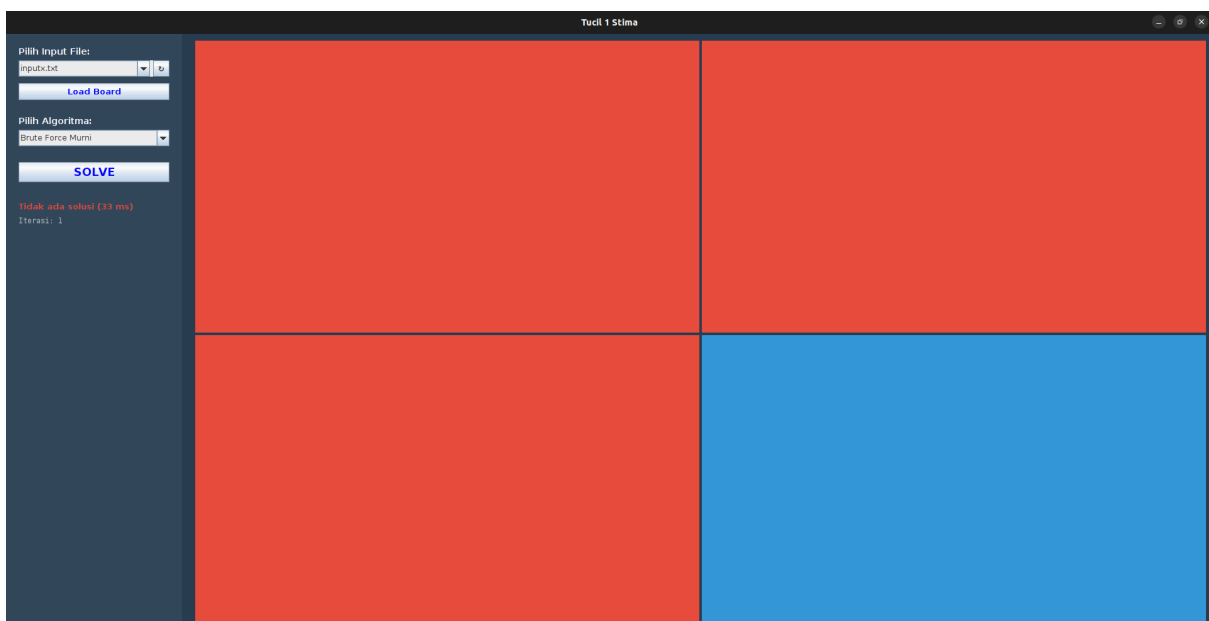


Gambar 3.1.2 Output Testcase normal

3. 2 TC 2 (Input tanpa solusi)

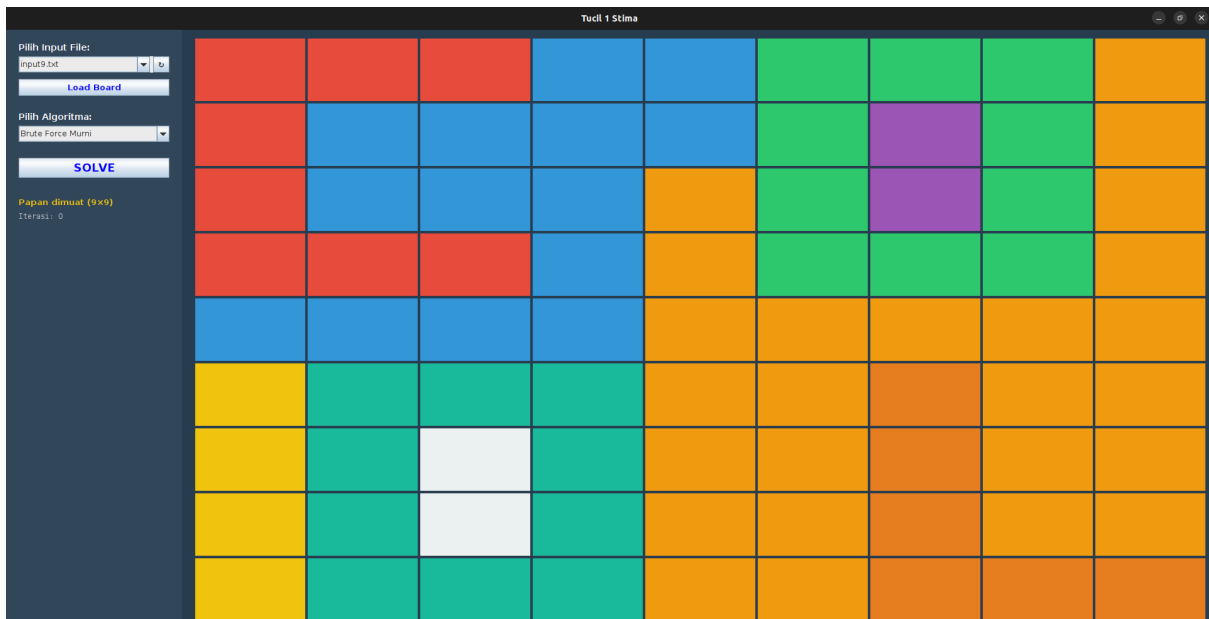


Gambar 3.2.1 Input Testcase tanpa solusi

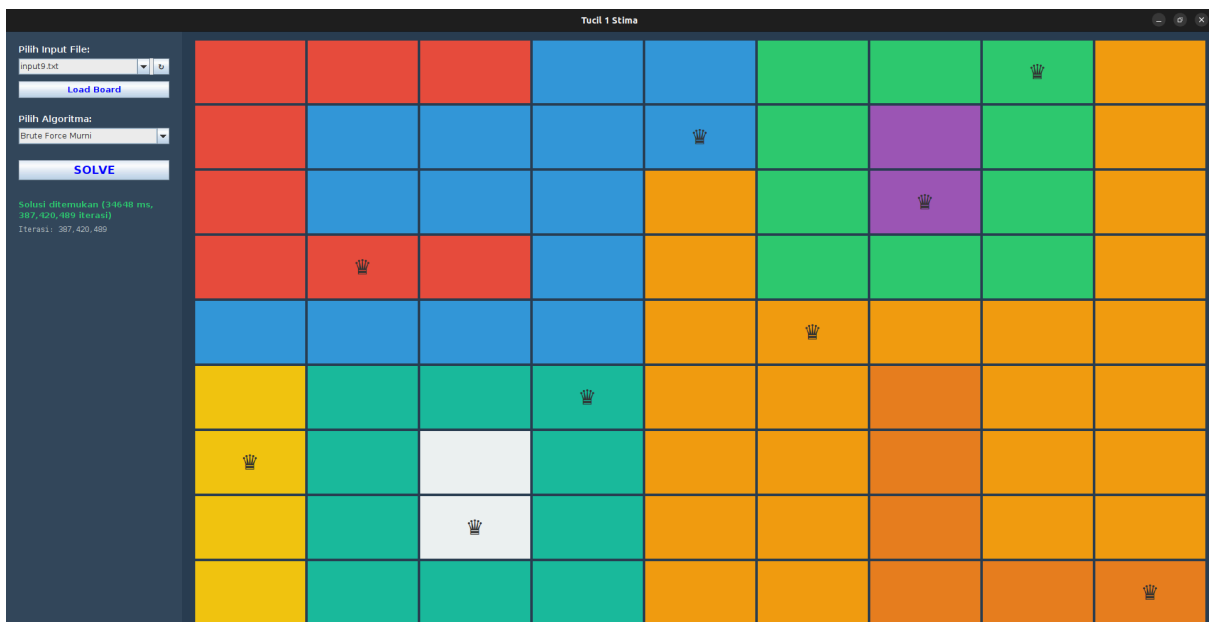


Gambar 3.2.2 Output Testcase tanpa solusi

3.3 TC3 (Input 9x9)

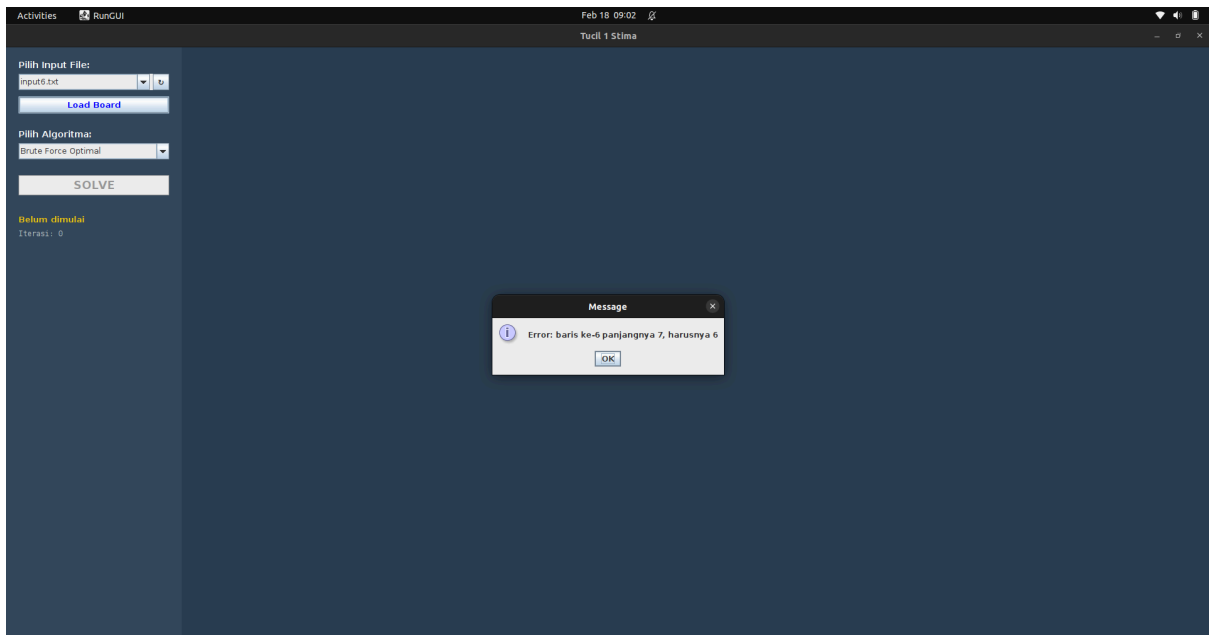


Gambar 3.3.1 Input Testcase 9x9



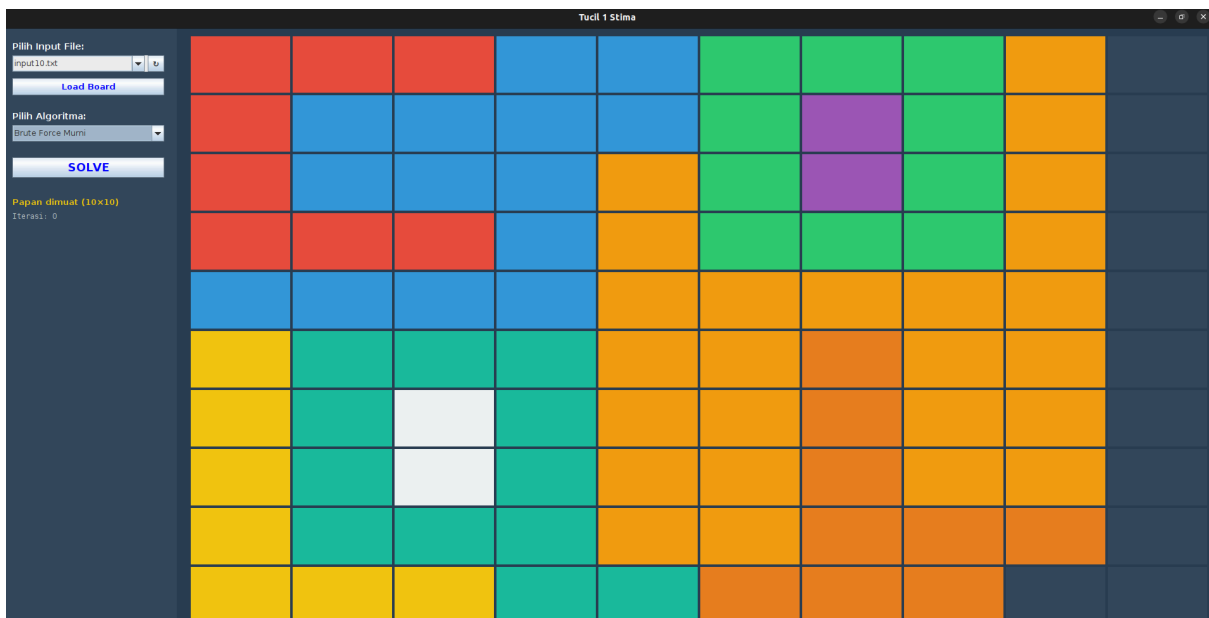
Gambar 3.3.2 Output Testcase 9x9

3.4 TC4 (Input tidak sama baris dan kolom)

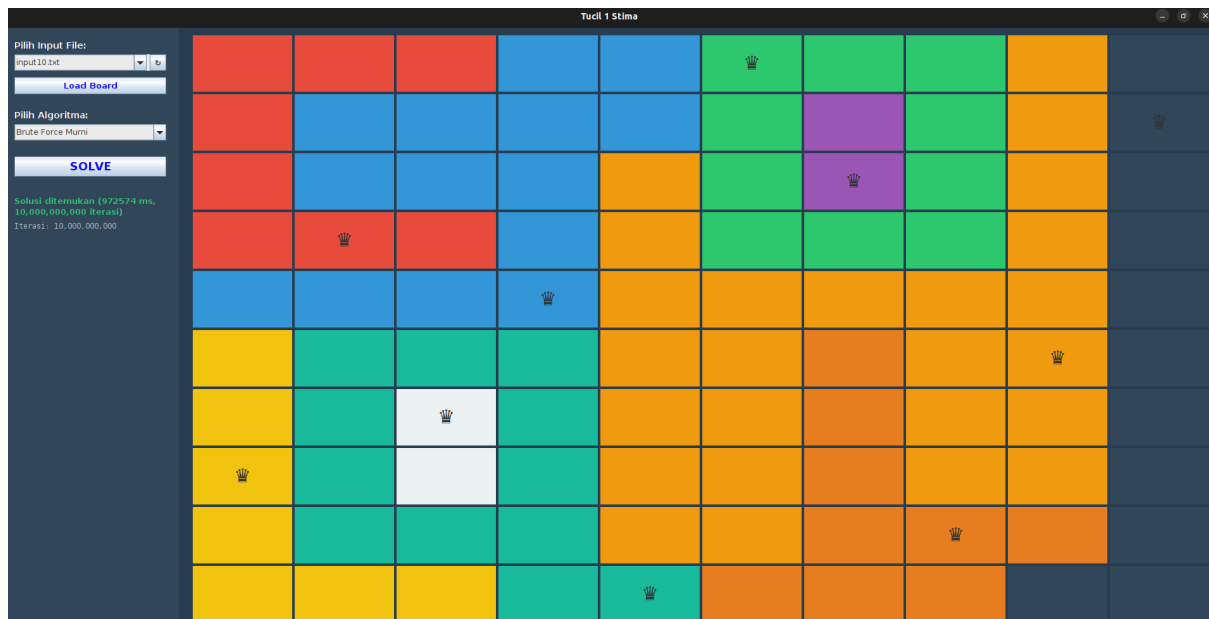


Gambar 3.4 Input dan Output Testcase 4

3.5 TC5 (10x10)



Gambar 3.5.1 Input Testcase 5



Gambar 3.5.2 output Testcase 5

4. Pranala repositori

https://github.com/Lloyd565/Tucil1_13524112

No.	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Program dapat menyimpan solusi dalam bentuk file gambar		✓

5. Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

A handwritten signature in black ink, consisting of a large, stylized 'R' followed by a horizontal line and a small flourish.

Richard Samuel Simanullang